

Installing Ollama on Windows 11

Local AI for the situation-report narration · default model mistral-nemo · ~15 minutes plus model download · no admin rights needed

1 Why local?

Ollama runs language models entirely on your own machine. For the situation report's AI narration this means: the delta briefing never leaves the system, and you need no account, no API key and no corporate approval for external services. This path shows up as `deployment_class "local"` in the AI banner and the operator-evidence log.

2 Prerequisites

- Windows 11, 64-bit; no administrator rights needed (Ollama installs into the user profile).
- Disk space: ~4 GB for Ollama itself plus the model — mistral-nemo downloads about 7 GB, mistral about 4 GB.
- Memory: 16 GB recommended for mistral-nemo (12B); with 8 GB pick the smaller mistral (7B).
- Without a GPU everything runs on the CPU — a narration then takes up to a minute; an NVIDIA or AMD GPU speeds things up noticeably and is used automatically.

3 Install

- Open ollama.com/download/windows in the browser and download OllamaSetup.exe.
- Double-click the file and follow the wizard — there are no choices to make.
- Ollama then runs in the background; the llama icon appears in the taskbar's notification area. It starts automatically after a reboot.

Verify the installation — open PowerShell or the command prompt (Windows key, type "powershell"):

```
ollama --version
```

4 Pull the model

The narration's default model is mistral-nemo (12B, good multilingual quality, management tone). The download happens once:

```
ollama pull mistral-nemo
```

```
# Alternative fuer Rechner mit 8 GB RAM / for 8 GB machines:
ollama pull mistral
```

Quick test directly in Ollama — the first answer takes longest because the model is loaded into memory:

```
ollama run mistral-nemo "Antworte mit einem Satz: Was ist ein Lagebild?"
# Chat beenden / leave the chat: /bye
```

5 Wiring check with SituationReport

In the repository or release folder, the providers list shows the discovered AI backends, and the test command runs a first labeled sample narration through Ollama:

```
python -m llm providers
python -m llm test
python -m llm test --lang en --model mistral
```

6 Use it

CLI: `--narrate` adds the "Narration (Entwurf)" section to the delta briefing — including the AI label, the numbers guard and the operator evidence (`llm_audit.jsonl` next to the output). GUI: in the Solutions & Portfolios window tick "AI narration (draft)", pick the provider next to it, then run "Delta briefing ..." as usual:

```
python -m portfolio --delta prev.json now.json --narrate --output delta.html --browser
```

```
# Anderes Modell / different model:
```

```
python -m portfolio --delta prev.json now.json --narrate --llm-model mistral --output delta.html
```

7 Troubleshooting

- "Python was not found" (pointing to the Microsoft Store): Windows redirects the python command to a Store placeholder. Run the commands from sections 5/6 in the repository folder and use `.venv\Scripts\python.exe` there instead of python (or activate once via `.venv\Scripts\Activate.ps1`) — alternatively `py -m llm test` with the Python launcher installed, or disable the app execution aliases `python.exe/python3.exe` (Settings > Apps > Advanced app settings).
- "Could not reach Ollama" / connection refused: Ollama is not running — start "Ollama" from the Start menu and wait for the llama icon in the notification area.
- "Ollama does not know model ...": the model is missing — run `ollama pull mistral-nemo` (section 4).
- Answers are slow: on a pure CPU up to a minute per narration is normal; mistral instead of mistral-nemo roughly halves the wait.
- Low space on C: — relocate the model folder:

```
# Modelle auf D: statt C: ablegen / store models on D:  
setx OLLAMA_MODELS "D:\ollama-models"  
# danach Ollama beenden und neu starten / then restart Ollama
```
- Quit: right-click the llama icon in the notification area → "Quit Ollama".

8 Privacy in one sentence

Ollama listens on your machine only (localhost:11434); neither briefing nor narration ever leaves the system — and the difference from the external path (Claude API) stays visible at all times via the `deployment_class` in banner and audit.